

# DBMS LAB – SQL JOINS AND TRANSACTIONS

YESHVITHA - 2520030194

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\yeshvitha> Get-ChildItem -Path C:\Users\yeshvitha -Directory -Recurse -Filter "DBSE_LAB" -ErrorAction SilentlyContinue
PS C:\Users\yeshvitha> cd "C:\Program Files\MySQL\MySQL Server 8.0\bin"
PS C:\Program Files\MySQL\MySQL Server 8.0\bin> .\mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE dbms_lab;
Database changed
mysql> USE dbms_lab;
Database changed
mysql> -- Q1. Insert a new employee
mysql> INSERT INTO employees
-> VALUES (108,'Arjun',1,101,52000);
Query OK, 1 row affected (0.07 sec)

mysql>
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+
| 101 | Rahul | 1 | NULL | 88000.00 |
| 102 | Anita | 1 | 101 | 66000.00 |
| 103 | Vijay | 2 | 101 | 55000.00 |
| 104 | Sneha | 2 | 103 | 45000.00 |
| 105 | Kiran | 3 | 101 | 70000.00 |
| 106 | Meena | 1 | 102 | 71500.00 |
| 107 | Arjun | 1 | 101 | 57200.00 |
| 108 | Arjun | 1 | 101 | 52000.00 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

```
mysql> -- Q2. Update salary of CSE employees by 10%
mysql> UPDATE employees
-> SET salary = salary * 1.10
-> WHERE dept_id = (
-> SELECT dept_id
-> FROM departments
-> WHERE dept_name = 'CSE'
-> );
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5 Changed: 5 Warnings: 0

mysql>
mysql> SELECT emp_name, salary
-> FROM employees;
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul | 96800.00 |
| Anita | 72600.00 |
| Vijay | 55000.00 |
| Sneha | 45000.00 |
| Kiran | 70000.00 |
| Meena | 78650.00 |
| Arjun | 62920.00 |
| Arjun | 57200.00 |
+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q3. Delete employees whose salary is below 30000
mysql> DELETE FROM employees
-> WHERE salary < 30000;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+
| 101 | Rahul | 1 | NULL | 96800.00 |
| 102 | Anita | 1 | 101 | 72600.00 |
| 103 | Vijay | 2 | 101 | 55000.00 |
| 104 | Sneha | 2 | 103 | 45000.00 |
| 105 | Kiran | 3 | 101 | 70000.00 |
| 106 | Meena | 1 | 102 | 78650.00 |
| 107 | Arjun | 1 | 101 | 62920.00 |
| 108 | Arjun | 1 | 101 | 57200.00 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

```
mysql> -- Q4. Display all employee details
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+
| 101 | Rahul | 1 | NULL | 96800.00 |
| 102 | Anita | 1 | 101 | 72600.00 |
| 103 | Vijay | 2 | 101 | 55000.00 |
| 104 | Sneha | 2 | 103 | 45000.00 |
| 105 | Kiran | 3 | 101 | 70000.00 |
| 106 | Meena | 1 | 102 | 78650.00 |
| 107 | Arjun | 1 | 101 | 62920.00 |
| 108 | Arjun | 1 | 101 | 57200.00 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q5. Display employee names along with department names
mysql> SELECT emp_name, dept_name
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Rahul | CSE |
| Anita | CSE |
| Vijay | ECE |
| Sneha | ECE |
| Kiran | MECH |
| Meena | CSE |
| Arjun | CSE |
| Arjun | CSE |
+-----+-----+
8 rows in set (0.01 sec)
```

```
mysql>
mysql> -- Q6. Display employees who do not belong to any department
mysql> SELECT emp_name
-> FROM employees
-> WHERE dept_id IS NULL;
Empty set (0.00 sec)

mysql>
mysql>
mysql> -- Q7. Display all departments even if they have no employees
mysql> SELECT dept_name, emp_name
-> FROM departments d
-> LEFT JOIN employees e
-> ON d.dept_id = e.dept_id;
+-----+-----+
| dept_name | emp_name |
+-----+-----+
| CSE | Arjun |
| CSE | Arjun |
| CSE | Meena |
| CSE | Anita |
| CSE | Rahul |
| ECE | Sneha |
| ECE | Vijay |
| MECH | Kiran |
| Civil | NULL |
+-----+-----+
9 rows in set (0.00 sec)
```

```
mysql> -- Q8. Display employee and manager names using SELF JOIN
mysql> SELECT e.emp_name AS Employee,
->         m.emp_name AS Manager
-> FROM employees e
-> LEFT JOIN employees m
-> ON e.manager_id = m.emp_id;
```

| Employee | Manager |
|----------|---------|
| Rahul    | NULL    |
| Anita    | Rahul   |
| Vijay    | Rahul   |
| Sneha    | Vijay   |
| Kiran    | Rahul   |
| Meena    | Anita   |
| Arjun    | Rahul   |
| Arjun    | Rahul   |

8 rows in set (0.00 sec)

```
mysql>
mysql>
mysql> -- Q9. Find the total number of employees
mysql> SELECT COUNT(*) AS TotalEmployees
-> FROM employees;
```

| TotalEmployees |
|----------------|
| 8              |

1 row in set (0.01 sec)

```
mysql> -- Q10. Find department-wise average salary
```

```
mysql> SELECT dept_name,  
->         AVG(salary) AS AverageSalary  
-> FROM employees e  
-> JOIN departments d  
-> ON e.dept_id = d.dept_id  
-> GROUP BY dept_name;
```

```
+-----+-----+  
| dept_name | AverageSalary |  
+-----+-----+  
| CSE       | 73634.000000 |  
| ECE       | 50000.000000 |  
| MECH      | 70000.000000 |  
+-----+-----+
```

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql>
```

```
mysql> -- Q11. Display departments where average salary exceeds 6  
0000
```

```
mysql> SELECT dept_name,  
->         AVG(salary) AS AverageSalary  
-> FROM employees e  
-> JOIN departments d  
-> ON e.dept_id = d.dept_id  
-> GROUP BY dept_name  
-> HAVING AVG(salary) > 60000;
```

```
+-----+-----+  
| dept_name | AverageSalary |  
+-----+-----+  
| CSE       | 73634.000000 |  
| MECH      | 70000.000000 |  
+-----+-----+
```

```
2 rows in set (0.00 sec)
```

```
mysql> -- Q12. Display the highest and lowest salary
mysql> SELECT MAX(salary) AS HighestSalary,
-> MIN(salary) AS LowestSalary
-> FROM employees;
```

```
+-----+-----+
| HighestSalary | LowestSalary |
+-----+-----+
|      96800.00 |      45000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql>
mysql>
mysql> -- Q13. Display employees earning more than the average salary
```

```
mysql> SELECT emp_name, salary
-> FROM employees
-> WHERE salary > (
-> SELECT AVG(salary)
-> FROM employees
-> );
```

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 96800.00 |
| Anita    | 72600.00 |
| Kiran    | 70000.00 |
| Meena    | 78650.00 |
+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql>
mysql>
mysql> -- Q14. Find employees working in the same department as Rahul
```

```
mysql> SELECT emp_name
-> FROM employees
-> WHERE dept_id = (
-> SELECT dept_id
```

```
mysql> -- Q20. Display the top-selling product
mysql> -- Cannot execute because Products and Order_Details
mysql> -- tables are not provided in the supplied manual.
mysql>
mysql>
mysql> -- Q21. Find departments with the maximum number of employ
```

```
ees
mysql> SELECT dept_name,
-> COUNT(*) AS EmployeeCount
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY dept_name
-> ORDER BY EmployeeCount DESC
-> LIMIT 1;
```

```
+-----+-----+
| dept_name | EmployeeCount |
+-----+-----+
| CSE      | 5             |
+-----+-----+
```

1 row in set (0.00 sec)

```
mysql>
mysql>
mysql> -- Q22. Display monthly sales using window functions
mysql> SELECT MONTH(order_date) AS Month,
-> SUM(total_amount) AS MonthlySales
-> FROM orders
-> GROUP BY MONTH(order_date);
```

```
+-----+-----+
| Month | MonthlySales |
+-----+-----+
| 8     | 9500.00      |
+-----+-----+
```

1 row in set (0.00 sec)

```
mysql> -- Q23. Rank customers based on total purchase amount
```

```
mysql> SELECT customer_name,  
->          SUM(total_amount) AS TotalPurchase,  
->          RANK() OVER (  
->            ORDER BY SUM(total_amount) DESC  
->          ) AS CustomerRank  
-> FROM customers c  
-> JOIN orders o  
-> ON c.customer_id = o.customer_id  
-> GROUP BY customer_name;
```

| customer_name | TotalPurchase | CustomerRank |
|---------------|---------------|--------------|
| Arun          | 5000.00       | 1            |
| Pooja         | 2500.00       | 2            |
| Ravi          | 2000.00       | 3            |

```
3 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql>
```

```
mysql> -- Q24. Display employees whose salary is greater than the  
ir manager's salary
```

```
mysql> SELECT e.emp_name AS Employee,  
->          e.salary AS EmployeeSalary,  
->          m.emp_name AS Manager,  
->          m.salary AS ManagerSalary  
-> FROM employees e  
-> JOIN employees m  
-> ON e.manager_id = m.emp_id  
-> WHERE e.salary > m.salary;
```

| Employee | EmployeeSalary | Manager | ManagerSalary |
|----------|----------------|---------|---------------|
| Meena    | 78650.00       | Anita   | 72600.00      |

```
1 row in set (0.00 sec)
```

```
mysql> -- Q25. Find duplicate email addresses using GROUP BY and  
HAVING
```

```
mysql> SELECT email,  
->          COUNT(*) AS Total  
-> FROM users  
-> GROUP BY email  
-> HAVING COUNT(*) > 1;
```

```
Empty set (0.06 sec)
```